Práctica 2: Sun RPC

Desarrollo de Sistemas Distribuidos. Grupo DSD-3.

Grado en Ingeniería Informática

Curso 2025-2026.

ETSIIT. Universidad de Granada.





UNIVERSIDAD
DE GRANADA

Autor:
Ángel Rodríguez Faya

Profesor:
José Ángel Segura Muros

Fecha de entrega:
13 de marzo de 2026

Índice

1. Introducción **2**

2. Diseño de la interfaz. Especificación del protocolo en el lenguaje RPC **3**

a. Definición del Programa. 4
b. Definición de Estructuras de Datos auxiliares. 4
c. Generación de archivos con *rpcgen* 5

3. Arquitectura del Servidor. **6**

a. `realizar_operacion(op1, op2, operador)` 12
b. Funciones intermediarias (*Wrappers*): las rutinas RPC 13
c. La gestión de los lotes. 13

4. Arquitectura Cliente. **13**

a. El programa principal: *main* 21
b. `mostrar_menu()` 22
c. `procesar_menu(opcion)` 22
d. `calcprog_1(*host, op1, op2, operador)` 22
e. `leer_lote(*lote, *array_aux)` 23
f. calcproglotes(*host, lote) 24

5. Ejemplos de compilación y ejecución. **24**

a. Compilación del proyecto 25
b. Ejecución del Proyecto 25
b.1. Puesta en marcha del Servidor 25
b.2. Ejecución del Cliente. 26

1. Introducción

El cometido de esta práctica es desarrollar una aplicación distribuida, en concreto, una calculadora basada en la arquitectura Cliente-Servidor. Para ello, usamos el paradigma de LLamadas a Procedimientos Remotos (*RPC, Remote Procedure Call*), utilizando el estándar *Sun RPC*, que nos permite generar código en lenguaje C de forma automática haciendo uso de *rpcgen*, una herramienta que integra este estándar.

Esta calculadora es un servicio que atiende peticiones de forma remota, realiza las operaciones que se le han pedido, y devuelve el resultado al cliente. En mi caso lo ejecuto todo de forma local, es decir, en mi propio ordenador aunque lo hago en terminarles distintas. Esto lo veremos en el punto **5. Ejemplos de compilación y ejecución**

Esta aplicación distribuida es capaz de realizar 11 operaciones separadas en tres grandes bloques:

- **Operaciones Básicas:** estas serían suma, resta, multiplicación y división de números flotantes (tipo: `double`), que nos permite tener mayor precisión.
- **Operaciones Avanzadas:** estas serían una serie de funciones científicas como potencias, logaritmos (en base 10 y neperianos), seno, coseno y tangente. También trabajan con `double` para una mayor precisión.
- **Operaciones en lote.** Nos permite introducir un lote de operaciones (del mismo tipo que las de los dos puntos anteriores) para enviárselas al servidor y que las calcule de golpe, y nos envíe los resultados. Con esto conseguimos optimizar la comunicación y reducir la sobrecarga del protocolo.

El usuario tiene dos opciones para poder introducir los datos:

1. Pasando los operandos y el operador como argumentos al llamar al programa (por terminal).
2. A través de un menú interactivo que se muestra al llamar al programa sin argumentos. Además, esta es la única forma de realizar el procesamiento de operaciones por lote, ya que por la forma anterior no se puede.

A continuación, veremos el diseño de la interfaz (la especificación del protocolo en el lenguaje RPC) y una breve explicación de su particular sintaxis. Después, veremos la arquitectura del servidor y la arquitectura del cliente. Terminaremos con unos ejemplos de compilación y ejecución.

2. Diseño de la interfaz. Especificación del protocolo en el lenguaje RPC

Para que podamos entablar una comunicación en Sun RPC necesitamos un archivo que defina una interfaz. Este archivo está escrito en lenguaje *XDR (eXternal Data Represetantation)*, que se encarga de establecer la forma de comunicarse el cliente y el servidor independientemente de la arquitectura de las máquinas, pues estas podrían ser distintas.

En mi implementación de esta práctica, este archivo es *calc.x*. Su código es el siguiente:

```
/* calc.x : Protocolo de calculadora */
/* Estas son las operaciones que va a ofrecer nuestro servidor */

/* Enumerado para nro de operaciones en el menú */
enum num_operaciones{
    OP_SUMA = 1,
    OP_RESTA = 2,
    OP_MULTIPLICACION = 3,
    OP_DIVISION = 4,
    OP_POTENCIA = 5,
    OP_LOG10 = 6,
    OP_LOGN = 7,
    OP_SENO = 8,
    OP_COSENO = 9,
    OP_TANGENTE = 10,
    OP_LOTES = 11
};

const MAX_OP = 30; /*Nro máximo de operaciones que se pueden calcular por lotes */

/* Encapsula los operandos y el operador en una sola estructura */
struct operacion{
    double op1; /* Operando 1 */
    char operador;
    double op2; /* Operando 2 */
};

typedef operacion operaciones<MAX_OP>; /* Operaciones a calcular por lotes */

typedef double resultados<MAX_OP>; /* Resultados de las operaciones por lotes */

program CALCPROG{
    version CALCVER {
        /* Operaciones en MODO BÁSICO */
        double SUMA(double, double) = 1;
```

```
36          double RESTA(double, double) = 2;
37          double MULTIPLICACION(double, double) = 3;
38          double DIVISION(double, double) = 4;
39
40          /* Operaciones en MODO AVANZADO */
41          double POTENCIA(double, double) = 5;
42          double LOG10(double) = 6;
43          double LOGN(double) = 7;
44          double SENO(double) = 8;
45          double COSENO(double) = 9;
46          double TANGENTE(double) = 10;
47          resultados CALCULAR_LOTES(operaciones) = 11;
48
49      } = 1;
50  } = 0x20000001;
```

Vamos a desglosarlo en dos partes fundamentales y verlas por separado.

a. Definición del Programa.

Lo encontramos en las líneas *(32-50)*. Esto es lo básico de un programa RPC, por eso lo explico antes. Encontramos el bloque *program CALCPROG*, que es el programa principal que contiene todas las funciones (o procedimientos) que el servidor expone y que el cliente puede invocar remotamente. Cada función tiene asignado un número identificador único y secuencial (del 1 al 12). Como podemos observar en el código, todos las operaciones devuelven un número de tipo flotante, para obtener una mayor precisión, y algunas sólo necesitan un solo parámetro de entrada. Están separadas como en dos bloques pues, realicé una primera implementación de la calculadora distribuida con estas 4 operaciones básicas, pero luego hice de nuevo otra implementación para añadirle el resto de operaciones.

Finalmente, encontramos el número programa (`CALCPROG = 0x20000001`) que lo identifica en la red de forma unívoca, y un número de versión (`CALCVER = 1`), permitiendo futuras actualizaciones del protocolo sin romper la compatibilidad con clientes antiguos.

b. Definición de Estructuras de Datos auxiliares.

Para poder realizar la funcionalidad de *operar en lote*, ha sido necesario definir estas estructuras de datos para que agrupen la información necesaria. En el código estarían entre las líneas *(1-31)*. Hemos definido:

- **Enumerado** `num_operaciones`: Identifica cada operación con un número, nos será útil más adelante cuando expliquemos el menú. En este caso, quería haber puesto los signos de las operaciones en lugar de números, pero XDR no realiza esa traducción a C, por lo que no me fue posible.
- **Constante** `MAX_OP`: Establece el número máximo de operaciones que se pueden realizar por lote.
- **Estructura** `operacion`: Son las unidades que almacena un lote, está formada por dos operadores y el operador.
- **Arrays estáticos** `operaciones` **y** `resultados`: El primer array almacena una serie de operaciones (instancias de la estructura operacion), que serán pasadas al servidor como un lote para que este realice las operaciones, y se las devuelva al cliente a través del segundo array.

c. Generación de archivos con *rpcgen*

. Llegados a este punto, tendremos que ejecutar la herramienta ***rpcgen*** para que nos genere las plantillas, stub y demás, que nos son necesarios para poder seguir avanzando en la práctica. Las opciones con las que se ejecuta este comando son:

- *-N*: Genera código estilo C con los argumentos pasados por valor.
- *-C*: Cógido ANSI-C o C++. Se usa junto con –N.
- *-a*: Genera todos los archivos plantilla.

Entonces, ejecutamos el comando:

```
rpcgen -NCa calc.x
```

Y podemos ver que se nos generan un montón de archivos, como cabeceras con definiciones comunes al cliente y servidor `*.h`, rutinas xdr para los tipos de datos definidos `*_xdr.c`, los stub del cliente y el servidor `*_svc.c` y `*_clnt.c` y las plantillas de los programas cliente y servidor `*_client.c` y `*_server.c`, que estas últimas son las que vamos a utilizar para los siguientes dos apartados.

3. Arquitectura del Servidor.

Como comentamos en el punto anterior, tras ejecutar *rpcgen* se nos genera el archivo plantilla del servidor. En mi implementación se llama `calc_server.c`, y su código es:

```c
/**
 * @file calc_server.c
 * @brief Servidor para la calculadora distribuida.
 *
 * @author Ángel Rodríguez Faya
 * @date 13-03-2026
 */

/*
 * This is sample code generated by rpcgen.
 * These are only templates and you can use them
 * as a guideline for developing your own functions.
 */

#include "calc.h"
#include <math.h>

/**
 * @brief Función para realizar la operación solicitada por el cliente.
 *
 * @param op1 El primer operando de la operación.
 * @param op2 El segundo operando de la operación (si la operación lo necesita,
 *                   si no lo necesita, se puede pasar cualquier valor, ya que no se utilizar
 * @param operador El operador de la operación (por ejemplo, '+', '-', 'x', '/', '^', 'L', 'l'
 * @return double El resultado de la operación solicitada por el cliente.
 */
double realizar_operacion(double op1, double op2, char operador){

        double resultado;

        switch (operador){

                /* Operaciones del MODO BÁSICO */

                case '+':
                        printf("Operación a realizar: Sumar %.3f + %.3f\n", op1, op2);
                        resultado = op1 + op2;
                        break;

                case '-':
                        printf("Operación a realizar: Restar %.3f - %.3f\n", op1, op2);
                        resultado = op1 - op2;
```

```
43                      break;
44
45              case 'x':
46                      printf("Operación a realizar: Multiplicar %.3f x %.3f\n", op1, op2);
47                      resultado = op1 * op2;
48                      break;
49
50              case '/':
51                      if (op2 == 0.0){
52                              printf("ERROR, no se ha podido realizar la operación : Dividir
53                              resultado = 0.0;
54                      } else {
55                              printf("Operación a realizar: Dividir %.3f / %.3f\n", op1, op2
56                              resultado = op1 / op2;
57                      }
58                      break;
59
60              /* Operaciones del MODO AVANZADO. */
61
62              case '^': /* Potencia */
63                      printf("Operación a realizar: Potencia %.2f ^ %.2f\n", op1, op2);
64                      resultado = pow(op1, op2); /*op1: base, op2: exponente*/
65                      break;
66
67              case 'L': /* Logaritmo en base 10*/
68                      printf("Operación a realizar: Logaritmo en base 10 de %.2f\n", op1);
69                      resultado = log10(op1);
70                      break;
71
72              case 'l': /* Logaritmo neperiano (en base e) */
73                      printf("Operación a realizar: Logaritmo neperiano de %.2f\n", op1);
74                      resultado = log(op1);
75                      break;
76
77              case 's': /* Seno */
78                      printf("Operación a realizar: Seno de %.3f grados\n", op1);
79                      resultado = sin(op1 * (M_PI/180.0)); /* Transformo a radianes*/
80                      break;
81
82              case 'c': /* Coseno */
83                      printf("Operación a realizar: Coseno de %.3f grados\n", op1);
84                      resultado = cos(op1 * (M_PI/180.0)); /* Transformo a radianes*/
85                      break;
86
87              case 't': /* Tangente */
88                      printf("Operación a realizar: Tangente de %.3f grados\n", op1);
89                      resultado = tan(op1 * (M_PI/180.0)); /* Transformo a radianes*/
90                      break;
91
```

```
92                  default:
93                          printf("Error: El operador %c no es válido.\n", operador);
94                          resultado = 0.0;
95                          break;
96          }
97
98          return resultado;
99  }
100
101 /**
102  * @brief Función para suma de dos números.
103  *
104  * @param arg1 El primer operando de la suma.
105  * @param arg2 El segundo operando de la suma.
106  * @param rqstp Un puntero a una estructura de tipo svc_req.
107  * @return double Un puntero a un double que contiene el resultado de la operación
108  */
109 double *
110 suma_1_svc(double arg1, double arg2,  struct svc_req *rqstp)
111 {
112         static double  result;
113
114         result = realizar_operacion(arg1, arg2, '+');
115
116         return &result;
117 }
118
119 /**
120  * @brief Función para resta de dos números.
121  *
122  * @param arg1 El primer operando de la resta.
123  * @param arg2 El segundo operando de la resta.
124  * @param rqstp Un puntero a una estructura de tipo svc_req.
125  * @return double Un puntero a un double que contiene el resultado de la operación
126  */
127 double *
128 resta_1_svc(double arg1, double arg2,  struct svc_req *rqstp)
129 {
130         static double  result;
131
132         result = realizar_operacion(arg1, arg2, '-');
133
134         return &result;
135 }
136
137
138 /**
139  * @brief Función para multiplicación de dos números.
140  *
```

```
141  * @param arg1 El primer operando de la multiplicación.
142  * @param arg2 El segundo operando de la multiplicación.
143  * @param rqstp Un puntero a una estructura de tipo svc_req.
144  * @return double Un puntero a un double que contiene el resultado de la operación
145  */
146 double *
147 multiplicacion_1_svc(double arg1, double arg2,  struct svc_req *rqstp)
148 {
149         static double  result;
150 
151         result = realizar_operacion(arg1, arg2, 'x');
152 
153         return &result;
154 }
155 
156 
157 /**
158  * @brief Función para división de dos números.
159  *
160  * @param arg1 El dividendo.
161  * @param arg2 El divisor.
162  * @param rqstp Un puntero a una estructura de tipo svc_req.
163  * @return double Un puntero a un double que contiene el resultado de la operación (El cocient
164  */
165 double *
166 division_1_svc(double arg1, double arg2,  struct svc_req *rqstp)
167 {
168         static double  result;
169 
170         result = realizar_operacion(arg1, arg2, '/');
171 
172         return &result;
173 }
174 
175 /**
176  * @brief Función para potencia de dos números.
177  *
178  * @param arg1 La base.
179  * @param arg2 El exponente.
180  * @param rqstp Un puntero a una estructura de tipo svc_req.
181  * @return double Un puntero a un double que contiene el resultado de la operación (La potenci
182  */
183 double *
184 potencia_1_svc(double arg1, double arg2,  struct svc_req *rqstp)
185 {
186         static double  result;
187 
188         result = realizar_operacion(arg1, arg2, '^');
189 
```

```
190         return &result;
191 }
192
193
194 /**
195  * @brief Función para logaritmo en base 10 de un número.
196  *
197  * @param arg1 El número del que se quiere calcular el logaritmo.
198  * @param rqstp Un puntero a una estructura de tipo svc_req.
199  * @return double Un puntero a un double que contiene el resultado de la operación
200  */
201 double *
202 log10_1_svc(double arg1,  struct svc_req *rqstp)
203 {
204         static double  result;
205
206         /* op2 queda anulado, pues no nos hará falta. */
207         result = realizar_operacion(arg1, 0.0, 'L');
208
209         return &result;
210 }
211
212
213 /**
214  * @brief Función para logaritmo neperiano (en base e) de un número.
215  *
216  * @param arg1 El número del que se quiere calcular el logaritmo.
217  * @param rqstp Un puntero a una estructura de tipo svc_req.
218  * @return double Un puntero a un double que contiene el resultado de la operación
219  */
220 double *
221 logn_1_svc(double arg1,  struct svc_req *rqstp)
222 {
223         static double  result;
224
225         /* op2 queda anulado, pues no nos hará falta. */
226         result = realizar_operacion(arg1, 0.0, 'l');
227
228         return &result;
229 }
230
231 /**
232  * @brief Función para seno de un número.
233  *
234  * @param arg1 El ángulo (en grados) del que se quiere calcular el seno.
235  * @param rqstp Un puntero a una estructura de tipo svc_req.
236  * @return double Un puntero a un double que contiene el resultado de la operación
237  */
238 double *
```

```
239 seno_1_svc(double arg1,  struct svc_req *rqstp)
240 {
241         static double  result;
242 
243         /* op2 queda anulado, pues no nos hará falta. */
244         result = realizar_operacion(arg1, 0.0, 's');
245 
246         return &result;
247 }
248 
249 /**
250  * @brief Función para coseno de un número.
251  *
252  * @param arg1 El ángulo (en grados) del que se quiere calcular el coseno.
253  * @param rqstp Un puntero a una estructura de tipo svc_req.
254  * @return double Un puntero a un double que contiene el resultado de la operación
255  */
256 double *
257 coseno_1_svc(double arg1,  struct svc_req *rqstp)
258 {
259         static double  result;
260 
261         /* op2 queda anulado, pues no nos hará falta. */
262         result = realizar_operacion(arg1, 0.0, 'c');
263 
264         return &result;
265 }
266 
267 /**
268  * @brief Función para tangente de un número.
269  *
270  * @param arg1 El ángulo (en grados) del que se quiere calcular la tangente.
271  * @param rqstp Un puntero a una estructura de tipo svc_req.
272  * @return double Un puntero a un double que contiene el resultado de la operación
273  */
274 double *
275 tangente_1_svc(double arg1,  struct svc_req *rqstp)
276 {
277         static double  result;
278 
279         /* op2 queda anulado, pues no nos hará falta. */
280         result = realizar_operacion(arg1, 0.0, 't');
281 
282         return &result;
283 }
284 
285 /**
286  * @brief Función para calcular un lote de operaciones solicitadas por el usuario.
287  *
```

```
288  * @param arg1 El lote de operaciones a calcular.
289  * @param rqstp Un puntero a una estructura de tipo svc_req.
290  * @return resultados Un puntero a una estructura de tipo resultados que contiene el resultado
291  */
292 resultados *
293 calcular_lotes_1_svc(operaciones arg1,  struct svc_req *rqstp){
294         static resultados result;
295 
296         static double resultados_aux[MAX_OP]; /*Array estático auxiliar para almacenar los res
297 
298         for (int i = 0; i < arg1.operaciones_len; i++){
299                 double op1 = arg1.operaciones_val[i].op1;
300                 double op2 = arg1.operaciones_val[i].op2;
301                 char operador = arg1.operaciones_val[i].operador;
302                 resultados_aux[i] = realizar_operacion(op1, op2, operador);
303         }
304         /* Ahora hacemos que result apunte al array auxiliar de resultados*/
305         result.resultados_val = resultados_aux;
306         result.resultados_len = arg1.operaciones_len; /* Ya que hay el mismo nro de operacione
307 
308         return &result;
309 }
```

Vamos a desglosarlo:

a. realizar_operacion(op1, op2, operador)

Es la función que hace de çerebro", pues es la que realiza las operaciones. En lugar de programar la lógica matemática en las funciones intermediarias (*Wrappers*), he decido centralizarlo todo en esta función y que e el resto de funciones (que son las que rpcgen les genera las cabeceras automáticamente) se apoyen en esta función central.

Se le pasan como parámetros los operandos y el operador, y mediante un switch-case con el operador se identifica la función que se desea realizar. Las funciones que sólo necesitan un operando, pasan el segundo operando como 0.0, aunque de todas formas si es el caso de una operación con un sólo operando el segundo ni se utiliza. A si que, no habría problemas de seguridad.

Las funciones que utiliza son de la librería `<math.h>` y devuelve el valor resultante de realizar la operación. Pensé en que la función recibiera como parámetro una estructura operación (ya que la tenía creada y me podía servir para aquí también), pero en las funciones *Wrappers* que veremos a continuación quedaba un código muy engorroso, ya que tenía que declarar la operación añadirle los operandos y el operador y pasársela a la función, luego esta función tenía que hacer lo inverso, extraer los operandos y el operador

de la estructura, y pues no quedaba realmente bien pues era hacer dos veces lo mismo.)

b. Funciones intermediarias (*Wrappers*): las rutinas RPC

Estas funciones son las que genera rpcgen automáticamente partiendo de las rutinas que definimos en el programa de RPC (*calc.x*). Estas funciones terminan en *_svc* como por ejemplo: *suma_1_svc*. Estas funciones cogen los dats que le llegan por la red y se lo pasan a la función central *realizar_operacion*, recogen el resultado, lo guardan en una variable static (es obligatorio en RPC para que la variable no se borre de la memoria al terminar la función) y devuelven el puntero al cliente.

c. La gestión de los lotes.

La función de los lotes (*calcular_lotes_1_svc*) es la más compleja. Recibe un array de operaciones y devuelve un array de resultados.

Para ello, vamos iterando sobre el array de peticiones (que es un array de estructuras operacion) extrayendo de cada operación los operandos y el operador, y llamando a la función central *realizar_operacion*. Los resultados devueltos por esta función los vamos almacenando en un array estático auxiliar. Finalmente, el array estático que devolverá esta función como resultado aún esta vacío, por lo que tenemos que indicarle su longitud y hacer que apunte al array auxiliar donde hemos estado almacenando los datos. De nuevo, recalcar la palabra static. Si no fuera estático, al terminar la función el servidor destruiría ese array, y le estaríamos enviando al cliente un puntero a una zona de memoria muerta (dando un error de Segmentation Fault o devolviendo ceros).

4. Arquitectura Cliente.

Ahora, cogemos la plantilla del cliente generada por *rpcgen* y la modificamos para implementar nuestro cliente tal y como se nos pide. En mi implementación se llama `calc_client.c`, y su código es:

```
1  /**
2   * @file calc_client.c
3   * @brief Cliente para la calculadora distribuida.
4   *
```

```
5   * @author Ángel Rodríguez Faya
6   * @date 13-03-2026
7   */
8
9   /*
10  * This is sample code generated by rpcgen.
11  * These are only templates and you can use them
12  * as a guideline for developing your own functions.
13  */
14
15  #include "calc.h"
16  #include <stdio.h>
17  #include <stdlib.h>
18
19  /**
20  * @brief Función para mostrar el menú de opciones al usuario.
21  *
22  * Esta función muestra un menú con las operaciones disponibles y solicita al usuario que elij
23  * Sólo está disponible en el modo interactivo, no en el modo terminal.
24  * @return int Devuelve la opción elegida por el usuario (un entero entre 1 y 11).
25  */
26  int mostrar_menu(){
27
28          int opcion = 0;
29
30          printf("\n==== CALCULADORA DISTRIBUIDA ===========\n");
31          printf("Operaciones del MODO BÁSICO: \n");
32          printf("1. Sumar                              '+'\n");
33          printf("2. Restar                             '-'\n");
34          printf("3. Multiplicar                        'x'\n");
35          printf("4. Dividir                            '/'\n");
36          printf("Operaciones del MODO AVANZADO: \n");
37          printf("5. Potencia                           '^'\n");
38          printf("6. Logaritmo en base 10               'L'\n");
39          printf("7. Logaritmo neperiano (base e)       'l'\n");
40          printf("8. Seno (valor input en grados)       's'\n");
41          printf("9. Coseno (valor input en grados)     'c'\n");
42          printf("10. Tangente (valor input en grados)'t'\n");
43          printf("11. Calcular un lote de operaciones.\n");
44          printf("=========================================\n");
45          do{
46                  printf("\nElige una opción (1-11): ");
47                  scanf("%d", &opcion);
48          } while (opcion < 1 || opcion > 11 );
49
50          return opcion;
51  }
52
53  /**
```

```
54  * @brief Función para procesar la opción elegida por el usuario en el menú y devolver el oper
55  *
56  * @param opcion La opción elegida por el usuario (un entero entre 1 y 11).
57  * @return char El operador correspondiente a la opción elegida por el usuario.
58  * Si la opción no es válida, se muestra un mensaje de error y se termina el programa.
59  */
60 char procesar_menu(int opcion){ /* Devuelve el operador deseado*/
61         char operador;
62
63         switch (opcion){
64                 case OP_SUMA: /* 1 */
65                         operador = '+';
66                         break;
67
68                 case OP_RESTA: /* 2 */
69                         operador = '-';
70                         break;
71
72                 case OP_MULTIPLICACION: /* 3 */
73                         operador = 'x';
74                         break;
75
76                 case OP_DIVISION: /* 4 */
77                         operador = '/';
78                         break;
79
80                 case OP_POTENCIA: /* 5 */
81                         operador = '^';
82                         break;
83
84                 case OP_LOG10: /* 6 */
85                         operador = 'L';
86                         break;
87
88                 case OP_LOGN: /* 7 */
89                         operador = 'l';
90                         break;
91
92                 case OP_SENO: /* 8 */
93                         operador = 's';
94                         break;
95
96                 case OP_COSENO: /* 9 */
97                         operador = 'c';
98                         break;
99
100                case OP_TANGENTE: /* 10 */
101                        operador = 't';
102                        break;
```

```
103
104                 default:
105                         printf("Error. La opción %d no es válida", opcion);
106                         exit(1);
107         }
108
109         return operador;
110 }
111
112 /**
113  * @brief Función para realizar una operación simple solicitada por el usuario.
114  * Se utiliza en el modo menú y en el modo terminal (pasando los parametros como argumentos de
115  *
116  * @param host El nombre del host donde se encuentra el servidor de la calculadora.
117  * @param op1 El primer operando de la operación.
118  * @param op2 El segundo operando de la operación (si la operación lo necesita,
119  *                   si no lo necesita, se puede pasar cualquier valor, ya que no se utilizará
120  * @param operador El operador de la operación (por ejemplo, '+', '-', 'x', '/', '^', 'L', 'l'
121  * @return void
122  */
123 void calcprog_1(char *host, double op1, double op2, char operador){
124
125         CLIENT *clnt;
126         double *result; /* Lo utilizaremos para el resultado de las operacion solicitada*/
127
128         #ifndef         DEBUG
129                 clnt = clnt_create (host, CALCPROG, CALCVER, "udp");
130                 if (clnt == NULL) {
131                         clnt_pcreateerror (host);
132                         exit (1);
133                 }
134         #endif          /* DEBUG */
135
136         /* Hacemos un switch-case para identificar la operación que ha pedido el usuario*/
137
138         switch (operador){
139                 case '+':
140                         result = suma_1(op1, op2, clnt);
141                         break;
142
143                 case '-':
144                         result = resta_1(op1, op2, clnt);
145                         break;
146
147                 case 'x': /* Se utiiza x en lugar de * porque si no, al leer desde la terminal
148                         result = multiplicacion_1(op1, op2, clnt);
149                         break;
150
151                 case '/':
```

```
152                         result = division_1(op1, op2, clnt);
153                         break;
154
155                 case '^':
156                         result = potencia_1(op1, op2, clnt);
157                         break;
158
159                 case 'L':
160                         result = log10_1(op1, clnt);
161                         break;
162
163                 case 'l':
164                         result = logn_1(op1, clnt);
165                         break;
166
167                 case 's':
168                         result = seno_1(op1, clnt);
169                         break;
170
171                 case 'c':
172                         result = coseno_1(op1, clnt);
173                         break;
174
175                 case 't':
176                         result = tangente_1(op1, clnt);
177                         break;
178
179                 default:
180                         printf("Error: El operador %c no es válido.\n", operador);
181                         exit(1);
182         }
183
184         /* Comprobamos que se ha realizado las oepracion correctamente*/
185         if (result == (double *) NULL) {
186         clnt_perror (clnt, "Error en la llamada al servidor");
187     } else {
188         printf("\nEl resultado de la operación es: %.3f\n", *result);
189     }
190
191         #ifndef        DEBUG
192                 clnt_destroy (clnt);
193         #endif         /* DEBUG */
194 }
195
196 /**
197  * @brief Función para leer un lote de operaciones desde la entrada estándar.
198  *
199  * @param lote Un puntero a una estructura de tipo operaciones donde se almacenarán las operac
200  * @param array_aux Un array auxiliar de tipo operacion donde se almacenarán temporalmente las
```

```
201  *                       antes de asignarlas a la estructura lote.
202  * @return void
203  */
204 void leer_lote(operaciones *lote, operacion *array_aux){
205 
206         int num_oper;
207         operacion oper;
208 
209         printf("\nIndique cuántas operaciones desea realizar por lote (máx %d): ", MAX_OP);
210         scanf("%d", &num_oper);
211 
212         /* Comprobamos que ha introducido un nro de veces correcto */
213         if(num_oper > MAX_OP) num_oper = MAX_OP;
214         if(num_oper < 1) num_oper = 1;
215 
216 
217         for(int i = 0; i < num_oper; i++){
218 
219                 printf("\n----- Introduzca la operación nro %d -------", i+1);
220 
221                 /* Leemos los operandos y el operador. */
222                 printf("\nIntroduzca el 1er operando: ");
223                 scanf("%lf", &oper.op1);
224 
225                 printf("Introduzca el operador (+ - x / ^ L l s c t): ");
226                 scanf(" %c", &oper.operador); /* Importante dejar un espacio para que nos e tr
227 
228                 /* Sólo leemos el segundo operando si la operación lo necesita. Si no lo neces
229                 if(oper.operador == '+' || oper.operador == '-' || oper.operador == 'x' ||
230                         oper.operador == '/' || oper.operador == '^'){
231                         printf("Introduzca el 2o operando: ");
232                         scanf("%lf", &oper.op2);
233                 } else {
234                         oper.op2 = 0.0;
235                 }
236 
237                 /* Lo guardamos en el array auxiliar */
238                 array_aux[i] = oper;
239 
240                 printf("------ Operación nro %d leida.--------------\n", i+1);
241         }
242 
243         printf("\n-----> El lote se ha leído correctamente <------\n");
244 
245 
246         /* Asignamos la memoria y el tamaño al lote,*/
247         lote->operaciones_val = array_aux;
248         lote->operaciones_len = num_oper;
249 }
```

```
250
251 /**
252  * @brief Función para calcular un lote de operaciones solicitadas por el usuario.
253  * @param host El nombre del host donde se encuentra el servidor de la calculadora.
254  * @param lote El lote de operaciones a calcular.
255  * @return void
256  */
257 void calcproglotes(char *host, operaciones lote){
258
259         CLIENT *clnt;
260         resultados *result; /* Puntero a un array de tipo resultados para ir almacenando los r
261
262         #ifndef DEBUG
263                 clnt = clnt_create(host, CALCPROG, CALCVER, "udp");
264                 if (clnt == NULL)
265                 {
266                         clnt_pcreateerror(host);
267                         exit(1);
268                 }
269         #endif /* DEBUG */
270
271         /* Llamamos a la función correspondiente apra que nos calcule el lote */
272         result = calcular_lotes_1(lote, clnt);
273
274         /* Comprobamos que se ha realizado las operaciones correctamente*/
275         if (result == (resultados *) NULL) {
276         clnt_perror (clnt, "Error en la llamada al servidor");
277     } else {
278         printf("\n=== RESULTADOS DEL CÁLCULO DEL LOTE ======\n");
279
280                 for(int i = 0; i < result->resultados_len; i++){
281                         /* lote y el array  de resultados (al que apunta result) tienen los mi
282                         double op1 = lote.operaciones_val[i].op1;
283                         char operador = lote.operaciones_val[i].operador;
284                         double op2 = lote.operaciones_val[i].op2;
285                         double resultado = result->resultados_val[i];
286
287                         switch (operador){
288
289                                 /* Suma, Resta, Multiplicación, División y Potencia se muestra
290                                 case '+':
291                                 case '-':
292                                 case 'x':
293                                 case '/':
294                                 case '^':
295                                         printf("%.3f %c %.3f = %.3f\n", op1, operador, op2, re
296                                         break;
297
298                                 case 'L':
```

```
299                                         printf("log10(%.3f) = %.3f\n", op1, resultado);
300                                         break;
301
302                                 case 'l':
303                                         printf("ln(%.3f) = %.3f\n", op1, resultado);
304                                         break;
305
306                                 case 's':
307                                         printf("sen(%.3f) = %.3f\n", op1, resultado);
308                                         break;
309
310                                 case 'c':
311                                         printf("cos(%.3f) = %.3f\n", op1, resultado);
312                                         break;
313
314                                 case 't':
315                                         printf("tan(%.3f) = %.3f\n", op1, resultado);
316                                         break;
317
318                         default:
319                                 break;
320                         }
321
322                 }
323
324                 printf("=========================================\n");
325     }
326
327         #ifndef DEBUG
328                 clnt_destroy(clnt);
329         #endif  /* DEBUG */
330 }
331
332
333
334 int main (int argc, char *argv[]) {
335         char *host;
336         double op1, op2; /* Operandos para las operaciones */
337         char operador;
338
339         if (argc == 5){ /* El usuario ha pasado los operandos por la terminal. */
340
341                 /* Extraemos los parámetros */
342                 host = argv[1]; /* Sería host = máquina */
343                 op1 = atof(argv[2]);
344                 operador = argv[3][0]; /*Se añade el [0] para que sólo se quede con el primer
345                 op2 = atof(argv[4]);
346
347                 /* Realizamos la operación simple */
```

```
348                 calcprog_1(host, op1, op2, operador);
349
350         } else if (argc == 2){ /* El usuario quiere que se le muestre el menú de opciones.*/
351
352                 host = argv[1];
353
354                 int opcion = mostrar_menu();
355
356                 if (opcion == CALCULAR_LOTES){
357                         operaciones lote;
358                         operacion array_aux[MAX_OP]; /* Array estático y auxiliar para almacen
359                         leer_lote(&lote, array_aux);
360                         calcproglotes(host, lote);
361
362                 } else { /* El usuario desea realizar alguna operación simple. */
363                         operador = procesar_menu(opcion);
364
365                         printf("\nIntroduzca el 1er operando: ");
366                         scanf("%lf", &op1);
367
368                         if(opcion == OP_SUMA || opcion == OP_RESTA || opcion == OP_MULTIPLICAC
369                                 opcion == OP_DIVISION || opcion == OP_POTENCIA){
370                                 printf("Introduzca el 2o operando: ");
371                                 scanf("%lf", &op2);
372                         }else{
373                                 op2 = 0.0;
374                         }
375
376                         /* Realizamos la operación simple */
377                         calcprog_1(host, op1, op2, operador);
378                 }
379
380         } else { /* El número de parámetros es incorrecto*/
381                 printf("Error al pasar los parámetros.\n");
382                 printf("Uso por terminal: %s <maquina> <numero> <operador> <numero>\n", argv[0
383                 printf("Uso por menú:     %s <maquina>\n", argv[0]);
384                 exit(1);
385                 }
386 }
```

Veamos cuál es la lógica que hay detrás de mi implementación:

a. El programa principal: *main*

Como ya comenté en la introducción, la principal característica del cliente son sus formas para recibir datos. Esto lo gestionamos en el *main*, comprobando el número de argumentos (argc):

- **Modo terminal**: Si el usuario pasa 5 argumentos (`ej. ./cliente localhost 2 + 3`), el programa los extrae usando atof (para convertir los char a double) y llama directamente a la función que se comunica con el servidor, que la explicaremos más adelante.

- **Modo interactivo (menú)**: Si el usuario solo pasa 2 argumentos (el ejecutable y la máquina, `ej. ./cliente localhost`), el programa lanza la función de *mostrar_menu*) para guiar al usuario paso a paso. Dependiendo de la opción del menú que seleccione el usuario se utilizarán una forma u otra. Por ejemplo, si selecciona la opción 11 que es la de calcular en lote (aquí es donde utilizo el enumerado que definí en *calc.x*, para encapsular estas palabras y no utilizar números mágicos en mi código). Entonces, si el usuario elige calcular por lotes se llama a las funciones correspondientes para procesar y calcular los lotes que son `leer_lote` y `calcproglotes` respectivamente. En cambio, si el usuario solicita hacer una sola operación, entonces se le pide el primer operando y el operador, y en función de si la operación necesita uno o dos operandos, entonces se le pide que introduzca al segundo o no. Finalmente, se llama a la función `calcprog_1` que es la encarga de llamar al servidor para que le devuelva el resultado de la operación.

b. `mostrar_menu()`

Como ya hemos comentado, esta función muestra el menú interactivo y devuelve un entero que codifica la opción escogida por el usuario.

c. `procesar_menu(opcion)`

Recibe como parámetro un entero que codifica la opción del menú escogida por el usuario y con un *switch-case* identifica la operación que se desea realizar, guardando en la variable `operador` el operador de la operación. Finalmente, la función devuelve ese operador, que es almacenado en una variable de tipo `char`.

d. `calcprog_1(*host, op1, op2, operador)`

Esta función es la encargada de realizar las operaciones simples y es la que hace que RPC cobre sentido, pues es la que reliza las llamadas al servidor. Recibe como parámetros el servidor (o *host)*), los dos operandos de la operación y el operador. Sus puntos clave son:

- Crea un cliente RPC con clnt_create usando protocolo UDP (que es más rápido y ligero) para realizar una conexión, y lo destruye al final con clnt_destroy para liberar recursos (desconexión).
- Con un switch-case: evalúa el operador y llama a la rutina stub específica (como suma_1 o seno_1).
- Tolerancia a fallos: comprueba si el resultado devuelto es NULL. En RPC, si el puntero es nulo, significa que hubo un error de red (el servidor está caído, la máquina no existe, etc.). Si no controlaramos esto, el cliente daría un Segmentation Fault al intentar leer un resultado que obviamente no ha llegado.

e. `leer_lote(*lote, *array_aux)`

Esta función es la encargada de interactuar con el usuario para recopilar y construir un lote (un bloque) completo de peticiones antes de enviarlas por la red. Recibe como parámetros un puntero al array dinámico `operaciones` (que será el lote que viaje al servidor) y un array auxiliar prereservado en memoria (al que apuntará *lote* una vez que hayamos terminado la lectura).

Sus puntos clave son los siguientes:

- **Control de límites:** Solicita al usuario el número de operaciones a realizar en el lote y lo acota de forma segura entre 1 y la constante `MAX_OP` para evitar posibles desbordamientos de memoria.
- **Gestión del buffer de entrada:** Utiliza un bucle para ir pidiendo iterativamente los operandos y el operador de cada operación. Un detalle técnico es el espacio en la función `scanf("%c", &oper.operador)`. Esto permite "limpiar el bufferïgnorando los espacios en blanco y evitando que el programa se trague el salto de línea (`\n`) residual de la lectura anterior.
- **Lectura condicional de operandos:** Mediante una estructura *if-else*, la función evalúa si el operador introducido corresponde a una operación con dos operandos (como suma o potencia). Si resulta ser una operación con un sólo operando (como seno o logaritmo),no pide un segundo operando al usuario y asigna un `0.0` por defecto, garantizando la consistencia de la estructura.
- Una vez que el `array_aux` está lleno con las instancias de las operaciones, se enlaza con la estructura XDR del lote. Para ello, se asigna

el puntero del array a `lote->operaciones_val` y la cantidad de elementos a `lote->operaciones_len`. Con este paso, los datos quedan empaquetados y listos para su transmisión.

f. calcproglotes(*host, lote)

Esta es la función encargada de realizar el procesamiento por lotes. Su objetivo es coger el array de operaciones *lote* que leímos y rellenamos anteriormente, enviarla al servidor mediante RPC y presentar los resultados al usuario de forma legible.

Sus puntos clave son los siguientes:

- **Gestión de la conexión RPC:** Al igual que en las operaciones simples, inicializa el cliente mediante la llamada `clnt_create` especificando el protocolo UDP, y se encarga de liberar los recursos de red al finalizar, mediante `clnt_destroy`.
- **Invocación y tolerancia a fallos:** Se realiza la llamada al *stub* generado por *rpcgen* (`calcular_lotes_1`). Tras la llamada, es se comprueba el puntero devuelto (`result == NULL`). Si la comunicación falla o el servidor se desconecta durante el cálculo del lote, esta comprobación evita una violación de segmento (*Segmentation Fault*) al intentar leer datos inexistentes, avisando al usuario del problema mediante `clnt_perror`.
- **Desempaquetado de array:** Si todo ha ido bien, la función recorre de forma paralela el array original de peticiones (`lote.operaciones_val`) y el array de respuestas devuelto por el servidor (`result->resultados_val`), utilizando la longitud devuelta (`result->resultados_len`) como límite. Dado que el servidor procesa y devuelve los datos secuencialmente, el índice i se corresponde exactamente en ambos arrays.
- **Mostrar la salida:** Para imprimir los resultados por pantalla, se utiliza un *switch-case* que evalúa el operador, porque en función del tipo de operación se muestra el resultado de una manera u otra (dependiendo de si la operación es binaria o unaria).

5. Ejemplos de compilación y ejecución.

Para concluir este documento, se detallará el proceso completo desde la generación de los ejecutables hasta la prueba de los diferentes modos de funcionamiento de la calculadora distribuida.

a. Compilación del proyecto

Gracias a la herramienta *rpcgen*, disponemos de un archivo `Makefile.calc` que automatiza la compilación de todos los archivos generados y de nuestro código fuente. Es importante que lo modifiquemos, pues necesitamos indicarle que debe compilar con unas librerías especificas para RPC, además de añadir la directiva **-lm** (necesaria para la librería matemática). Por tanto, nos vamos al Makefile y buscamos estas dos líneas:

```
CFLAGS +=
```

```
LDLIBS +=
```

Y les añadimos las siguientes opciones:

```
CFLAGS += -g -I/usr/include/tirpc
```

```
LDLIBS += -ltirpc -lm
```

la compilación se lanza con el siguiente comando en la terminal

```
make -f Makefile.calc
```

Si no hay errores de sintaxis, este comando invocará al compilador `gcc` y generará dos archivos ejecutables independientes: `calc_server` y `calc_client`.

b. Ejecución del Proyecto

b.1. Puesta en marcha del Servidor

El primer paso es levantar el servicio. Para ello, en una terminal independiente (o en segundo plano), ejecutamos el servidor. Al ser un servicio de red, se quedará a la escucha en el puerto asignado por el *portmapper* del sistema operativo esperando las peticiones RPC:

```
./calc_server
```

En la siguiente imagen se muestra un ejemplo donde se compila el proyecto y se lanza el servidor:

```
angelrf@angel-ubuntu:~/Practica_2_Llamada_a_procedimiento_re
moto_RPC/codigo$ make -f Makefile.calc
cc -g -I/usr/include/tirpc    -c -o calc_clnt.o calc_clnt.c
cc -g -I/usr/include/tirpc    -c -o calc_client.o calc_clien
t.c
cc -g -I/usr/include/tirpc    -c -o calc_xdr.o calc_xdr.c
cc -g -I/usr/include/tirpc     -o calc_client  calc_clnt.o c
alc_client.o calc_xdr.o -ltirpc -lm
cc -g -I/usr/include/tirpc    -c -o calc_svc.o calc_svc.c
cc -g -I/usr/include/tirpc    -c -o calc_server.o calc_serve
r.c
cc -g -I/usr/include/tirpc     -o calc_server  calc_svc.o ca
lc_server.o calc_xdr.o -ltirpc -lm
angelrf@angel-ubuntu:~/Practica_2_Llamada_a_procedimiento_re
moto_RPC/codigo$ ./calc_server
```

b.2. Ejecución del Cliente.

Con el servidor a la escucha, abrimos una segunda terminal para actuar como cliente. Recordamos que teníamos dos modos de funcionamiento, veamoslos por separado:

- **Modo Terminal.** El primer modo de funcionamiento permite realizar cálculos rápidos pasando los parámetros directamente en la llamada al programa. la forma de llamarlo es:

  `./calc_client maquina numero operador numero`

  Los operadores pueden ser: $+ \ - \ x \ / \ L \ l \ s \ c \ t$

  En la siguiente imagen se muestra un ejemplo donde se calcula la potencia (2^8):

```
angelrf@angel-ubuntu:~/Practica_2_Llamada_a_procedimiento_re
moto_RPC/codigo$ ./calc_client localhost 2 ^ 8

El resultado de la operación es: 256.000
```

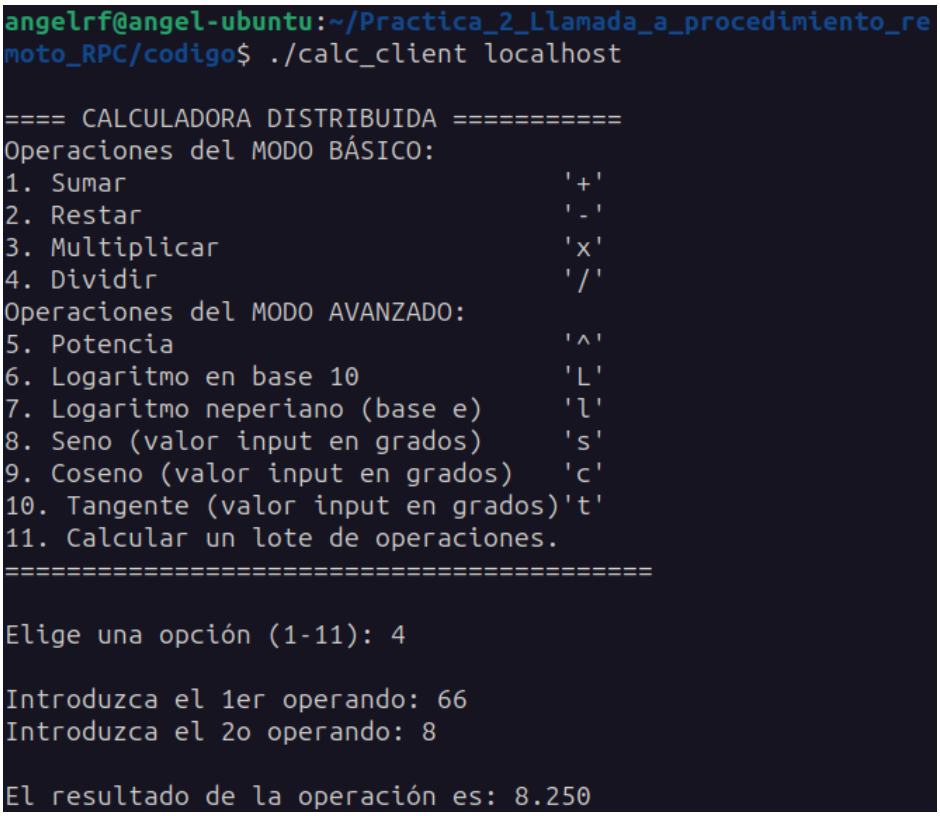
- **Menú interactivo y Lotes.** Si el usuario desea una experiencia más guiada o necesita realizar múltiples cálculos a la vez de forma eficiente, invocará al cliente indicando únicamente la máquina servidor:

  `./calc_client localhost`

  Al hacerlo, el programa desplegará el menú interactivo. Si el usuario selecciona la opción correspondiente a las operaciones por lote, se activará la función `leer_lote`, solicitando los datos de forma secuencial.

Una vez recopilados, viajarán por la red en una única transacción y el cliente mostrará los resultados formateados.

En la siguiente imagen se muestra un ejemplo del menú interactivo, junto a una petición de realizar una división (66 / 8):



Para finalizar, se muestra un ejemplo del calculo de un lote compuesto por 3 operaciones:

